java : Module 2

▼ Inheritance

I**nheritance in Java** is a mechanism in which one object acquires all the properties and behaviors of a parent object. It is an important part of OOPs (Object Oriented programming system).

In Java, it is possible to inherit attributes and methods from one class to another. We group the "inheritance concept" into two categories:

- **subclass** (child) - the class that inherits from another class
- **superclass** (parent) - the class being inherited from

To inherit from a class, use the `extends` keyword.

▼ Creating Sub-classes

```
class Subclass-name extends Superclass-name
{
   //methods and fields
}
```

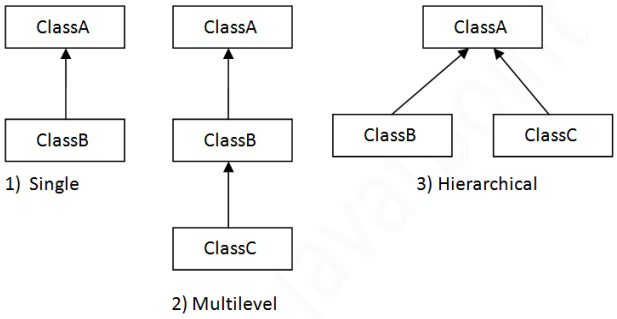
Eg: inherting a subclass (single inheritance)

```
//This is a superclass
class Employee {
    void salary() {
        System.out.println("Salary= 200000");
    }
}
//This is a subclass
class Programmer extends Employee {
// Programmer class inherits from Employee class
    void bonus() {
        System.out.println("Bonus=50000");
    }
}


class single_inheritance {
    public static void main(String args[]) {
        Programmer p = new Programmer();
```

```
        p.salary(); // calls method of super class
        p.bonus(); // calls method of sub class
    }
}
```

▼ Types of java Inheritance

On the basis of class, there can be three types of inheritance in java: single, multilevel and hierarchical.In java programming, multiple and hybrid inheritance is supported through interface only.



▼ Single Inheritance

When a class inherits another class, it is known as a *single inheritance.*

```
class Animal{

void eat(){
System.out.println("eating...");
}
}

class Dog extends Animal{
void bark(){
System.out.println("barking...");
}
}

class TestInheritance{
public static void main(String args[]){
Dog d=new Dog();
d.bark();
d.eat();
}
}
```

Output:

```
barking...
eating...
```

Dog class inherits the Animal class, so there is the single inheritance.

▼ Multilevel Inheritance

The multi-level inheritance includes the involvement of at least two or more than two classes. One class inherits the features from a parent class and the newly created sub-class becomes the base class for another new class.

```
Class SuperClass
{
```

```
   public void methodA()
   {
     System.out.println("SuperClass");
   }
}


Class SubClassB extends SuperClass
{
   public void methodB()
   {
     System.out.println("SubClassB ");
   }
}



Class SubClassC extends SubClassB
{
   public void methodC()
   {
     System.out.println("SubClassC");
   }


   public static void main(String args[])
   {
     SubClassB obj = new SubClassB();
     SubClassB.methodA(); //calling super class method
     SubClassB.methodB(); //calling subclass 1 method
     SubClassB.methodC(); //calling own method
  }
```

Output

```
SuperClass

SubClassB

SubClassC
```

▼ Hierarchical Inheritance

"**Hierarchical inheritance**" occurs when **multiple child classes**
inherit the methods and properties of the same parent class. This simply means we have only one super-class and multiple sub-classes in hierarchical inheritance in Java.

```
// creating the base class(or superclass)
class BaseClass
{
  int parentNum = 10;
}

// creating the subclass1 that inherits the base class
class SubClass1 extends BaseClass
{
  int childNum1 = 1;
}

// creating the subclass2 that inherits the base class
class SubClass2 extends BaseClass
{
  int childNum2 = 2;
}

// creating the subclass3 that inherits the base class
class SubClass3 extends BaseClass
{
  int childNum3 = 3;
```

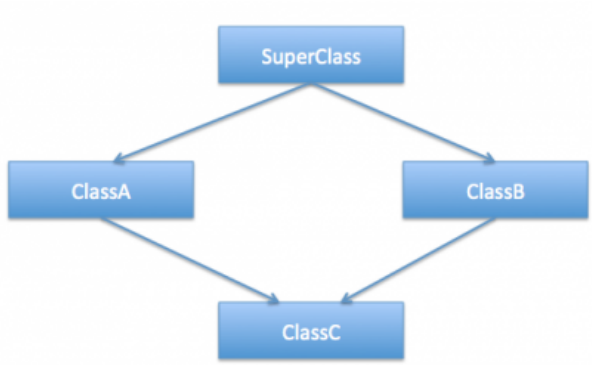
```
}
public class Main
{
  public static void main(String args[])
  {
    SubClass1 childObj1 = new SubClass1 ();
    SubClass2 childObj2 = new SubClass2 ();
    SubClass3 childObj3 = new SubClass3 ();

    System.out.println("parentNum * childNum1 = " + childObj1.parentNum * childObj1.childNum1); // 10 * 1 = 10
    System.out.println("parentNum * childNum2 = " + childObj2.parentNum * childObj2.childNum2); // 10 * 2 = 20
    System.out.println("parentNum * childNum3 = " + childObj3.parentNum * childObj3.childNum3); // 10 * 3 = 30
  }
}
```

output

```
parentNum * childNum1 = 10
parentNum * childNum2 = 20
parentNum * childNum3 = 30
```

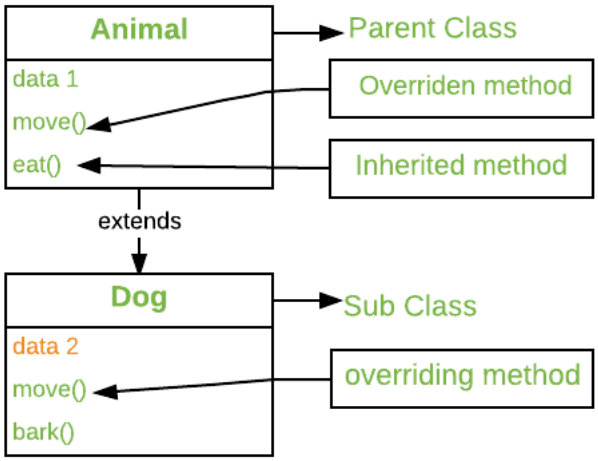
▼ *Q) Why multiple inheritance is not supported in java?*

java doesn't provide support for multiple inheritance in classes. Java doesn't support multiple inheritances in classes because it can lead to **diamond problem** and rather than providing some complex way to solve it, there are better ways through which we can achieve the same result as multiple inheritances or via implementing interfaces.



▼ Method overriding

Overriding is a feature that allows a subclass or child class to provide a specific implementation of a method that is already provided by one of its super-classes or parent classes. When a method in a subclass has the same name, same parameters or signature, and same return type(or sub-type) as a method in its super-class, then the method in the subclass is said to *override* the method in the super-class.




```
class Animal {
   public void displayInfo() {
      System.out.println("I am an animal.");
   }
}

class Dog extends Animal {

   public void displayInfo() {
      //Here the oveririding occurs
      System.out.println("I am a dog.");
   }
}

public class Main {
   public static void main(String[] args) {
      Dog d1 = new Dog();
      d1.displayInfo();
   }
}
```

Java Overriding Rules

- Both the superclass and the subclass must have the same method name, the same return type and the same parameter list.
- We cannot override the method declared as `final` and `static`.
- We should always override abstract methods of the superclass.

▼ Super Keyword

`Super` keyword is the alternative of `virtual` keyword in cpp

`Super` keyword is introduced in java for accesssing the superclass methods after overriding .

```
class Animal {
   public void displayInfo() {
      System.out.println("I am an animal from super class.");
   }
}

class Dog extends Animal {
  //This function is first overrided from the superclass
   public void displayInfo() {
     //calling the exact same function but it is mentioned in the super class
     super.displayInfo();
      System.out.println("I am a dog from subclass.");
   }
}
```

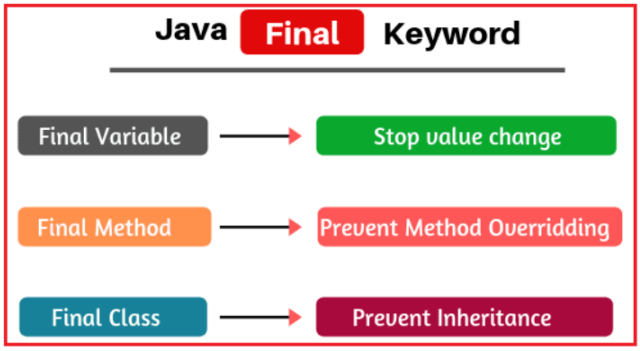
```
class Main {
   public static void main(String[] args) {
      Dog d1 = new Dog();
      d1.displayInfo();
   }
}
```

Output

```
I am an animal from super class.
I am a dog from subclass.
```

▼ Final keyword

https://youtu.be/WZgcRSWVgpQ



While inheritance enables us to reuse existing code, sometimes we do need to **set limitations on extensibility** for various reasons; the `final` keyword allows us to do exactly that.

▼ Final Classes

Classes marked as `final` can't be extended.

Eg:

```
public final class Cat {

    private int weight;

    // standard getter and setter
}
public class BlackCat extends Cat {
}
//here we used are trying to inherit the supercalss with final keyword
```

output

```
The type BlackCat cannot subclass the final class Cat
```

Here the Java compiler restricted the class inhertance due to the consideration of the `Final` keyword .

▼ Final variables

Variables marked as *final* can't be reassigned. Once a `final` variable is initialized, it can’t be altered.

```
class Bike9{
 final int speedlimit=90;//final variable
 void run(){
  speedlimit=400;
 }
 public static void main(String args[]){
 Bike9 obj=new  Bike9();
 obj.run();
 }
```

```
Output:Compile Time Error
```

▼ Final Methods

Methods marked as *final* cannot be overridden.

When we design a class and feel that a method shouldn’t be overridden, we can make this method `final`

```
public class Dog {
    public final void sound() {
        // ...
    }
}
```

Inherting this class and overriding the method `sound()`

```
public class BlackDog extends Dog {
    public void sound() {
    }
}
```

output

- `Cannot override the final method from Dog`
  `sound() method is final and can’t be overridden`

Here the compiler stated that we cant override the methods with `Final` keyword.

▼ Packages and Interfaces

▼ packages

https://youtu.be/xd_pRY_SYKg

A Java package is a collection of similar types of sub-packages, interfaces, and classes.
In Java, there are two types of packages:

- built-in packages
- user-defined packages.

The package keyword is used in Java to create Java packages.

Many in-built packages are available in Java, including util, lang, awt, javax, swing, net, io, sql, etc. We can import all members of a package using packagename.* statement.

Advantages of Java Packages

- Java package is used to categorize the classes and interfaces so that they can be easily maintained.
- Java package provides access protection.
- Java package removes naming collisions.

Accessing packages

There are three ways to access the package from outside the package.

▼ Using `package name.*`

If you use package.* then all the classes and interfaces of this package will be accessible but not subpackages.
The import keyword is used to make the classes and interface of another package accessible to the current package.

Example

```
//Here we are creating a package using the below line of code
package pack;
public class A{
public void msg(){System.out.println("Hello");}
}
//This file is saved as A.java
```

Importing the Above mentioned package ‘pack’ into another java program.

```
//We are also creatinga package 'mypack' here.
package mypack;//importing the user defined pacakge of above
import pack.*;
class B{
public static void main(String args[]){
A obj = new A();
obj.msg();
}
}
```

▼ Using `package name .class`

If you import package.classname then only declared class of this package will be accessible.

Example

```
package pack;
public class A{
public void msg(){System.out.println("Hello");}
}
```

using `import pack.A;`

```
package mypack;
import pack.A;
class B{
public static void main(String args[]){
A obj = new A();
obj.msg();
}
}
```

▼ **Using fully qualified name**

If you use fully qualified name then only declared class of this package will be accessible. Now there is no need to import. But you need to use fully qualified name every time when you are accessing the class or interface.

It is generally used when two packages have same class name e.g. java.util and java.sql packages contain Date class.

Example

```
package pack;
public class A{
public void msg(){System.out.println("Hello");}
}


//save as A.java
```

```
package mypack;
class B{
public static void main(String args[]){
pack.A obj = new pack.A();//using fully qualified name
obj.msg();
}
}
```

▼ Access Modifiers

The access modifiers in Java specifies the accessibility or scope of a field, method, constructor, or class. We can change the access level of fields, constructors, methods, and class by applying the access modifier on it.

Access Modifiers in Java are :

▼ Private

The private access modifier is specified using the keyword **private**.

- The methods or data members declared as private are accessible only **within the class** in which they are declared.
- Any other **class of** the **same package will not be able to access** these members.
- Top-level classes or interfaces can not be declared as private because
    1. private means “only visible within the enclosing class”.
    2. protected means “only visible within the enclosing class and any subclasses”

Example

```
// Java program to illustrate error while
// using class from different package with
// private modifier
package p1;

class A
{
private void display()
  {
    System.out.println("Hello World !");
  }
}

class B
{
public static void main(String args[])
  {
    A obj = new A();
    // Trying to access private method
    // of another class
    obj.display();
  }
}
```

Output

```
error: display() has private access in A
        obj.display();
```

▼ Default

The **access level** of a default modifier is **only within the package.** It cannot be accessed from outside the package. If you do not specify any access level, it will be the default.

Example

```
// Java program to illustrate default modifier
package p1;

// Class Geeks is having Default access modifier
class Geek
{
  void display()
  {
    System.out.println("Hello World!");
  }
}
```

```
// Java program to illustrate error while
// using class from different package with
// default modifier
```

```
package p2;
import p1.*;

// This class is having default access modifier
class GeekNew
{
  public static void main(String args[])
  {
    // Accessing class Geek from package p1
    Geeks obj = new Geek();

    obj.display();
  }
}
```

Output

`Compile Time error`

▼ Protected

The protected access modifier is specified using the keyword **protected**.

- The methods or data members declared as protected are **accessible within the same package or subclasses in different packages.**

Example

```
// Java program to illustrate
// protected modifier
package p1;

// Class A
public class A
{
protected void display()
  {
    System.out.println("hello World");
  }
}
```

```
// Java program to illustrate
// protected modifier
package p2;
import p1.*; // importing all classes in package p1

// Class B is subclass of A
class B extends A
{
public static void main(String args[])
{
  B obj = new B();
  obj.display();
}

}
```

Output:

```
hello World
```

▼ Public

The public access modifier is specified using the keyword **public**.

- The public access modifier has the **widest scope** among all other access modifiers.
- Classes, methods, or data members that are declared as public are **accessible from everywhere** in the program. There is no restriction on the scope of public data members.

Example

```
// Java program to illustrate
// public modifier
package p1;
public class A
{
public void display()//here we used the public keyword
  {
    System.out.println("HelloWorld");
  }
}
```

```
package p2;
import p1.*;
class B {
  public static void main(String args[])
  {
    A obj = new A();
    obj.display();
  }
}
```

Output

`HelloWorld`

Access Modifier	within class	within package	outside package by subclass only	outside package
Private	Y	N	N	N
Default	Y	Y	N	N
Protected	Y	Y	Y	N
Public	Y	Y	Y	

▼ Interfaces

▼ What is an interface in java ?

In Java, an interface is a collection of **abstract methods** and constant variables that define a set of behaviors that a class can implement. Interfaces allow you to specify what a class should do, but not how it should do it.

Here is an example of an interface in Java:

```
public interface Animal {
  public void makeSound();
  public void move();
}
```

in this example, the `Animal` interface defines two abstract methods: `makeSound` and `move`. These methods do not have any implementation, they just specify that any class that implements the `Animal` interface must have these two methods.

To implement an interface in a class, you use the `implements` keyword, like this:

```
public class Dog implements Animal {
  public void makeSound() {
    System.out.println("Bark!");
  }
  public void move() {
    System.out.println("Walking on four legs.");
  }
}
```

In this example, the `Dog` class implements the `Animal` interface and provides an implementation for the two abstract methods defined in the interface.

Interfaces are a useful feature in Java because they allow you to define a set of behaviors that a class must implement, without having to specify how those behaviors are implemented. This allows you to create flexible and reusable code.

▼ Uses of java interfaces

There are mainly three reasons to use interface. They are

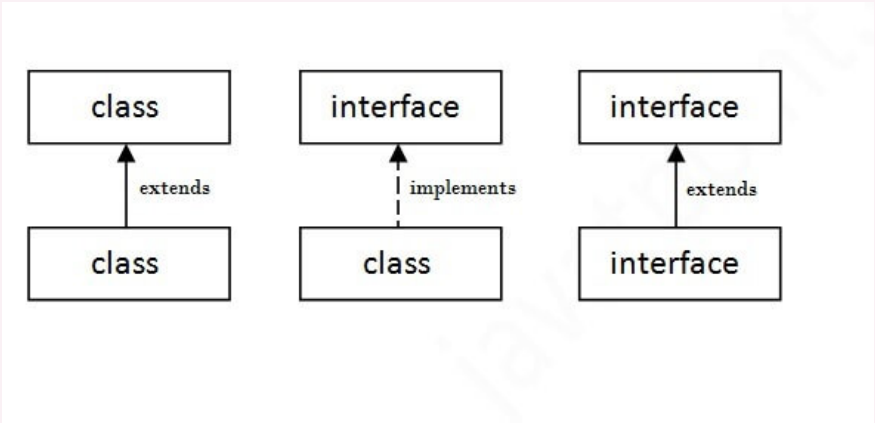
- It is used to achieve abstraction.
- By interface, we can support the functionality of multiple inheritance.
- It can be used to achieve loose coupling.

▼ Declaration of interfaces

To declare an interface in Java, you use the `interface` keyword, followed by the name of the interface and a set of abstract methods. An abstract method is a method that is declared in an interface or abstract class but does not have an implementation.

```
interface <interface_name>{

    // declare constant fields
    // declare methods that abstract
    // by default.
}
```

▼ Relationship between classes and Inheritance



1. A class can implement an interface: When a class implements an interface, it must provide an implementation for all the abstract methods declared in the interface. This allows the class to conform to the contract defined by the interface and use the interface's methods. To implement an interface in a class, you use the `implements` keyword, like this:

```
public class Dog implements Animal {
  public void makeSound() {
    System.out.println("Bark!");
  }
  public void move() {
    System.out.println("Walking on four legs.");
  }
}
```

2. A class can extend another class and implement one or more interfaces: In Java, **a class can only inherit from one super-class, but it can implement multiple interfaces.** This allows you to reuse code from multiple sources and create classes with a variety of behaviors. To extend a class and implement an interface, you use the `extends` and `implements` keywords, like this:

```
public class Cat extends Pet implements Animal {
  public void makeSound() {
    System.out.println("Meow!");
  }
  public void move() {
    System.out.println("Walking on four legs.");
  }
}
```

3. An interface can extend one or more interfaces: An interface can extend one or more interfaces to inherit their abstract methods and constant variables. This allows you to create a hierarchy of interfaces and reuse code in a structured way. To extend an interface, you use the `extends` keyword, like this:

```
public interface DomesticAnimal extends Animal {
  public void beDomestic();
}
```

In this example, the `DomesticAnimal` interface extends the `Animal` interface and inherits its abstract methods and constant variables. It also declares a new abstract method called `beDomestic`.

Overall, the relationship between classes and interfaces in Java allows you to create flexible and reusable code, and it enables a variety of design patterns and programming techniques.

▼ IO packages

https://www.youtube.com/playlist?list=PLfu_Bpi_zcDO4CdNYNS2Wten1vLuQfgp7

Java I/O (Input and Output) is used *to process the input* and *produce the output*.

Java uses the concept of a stream to make I/O operation fast. The `java.io` package contains all the classes required for input and output operations.

We can perform **file handling in Java** by Java I/O API.

What can you do with I/O

- Reading and writing files
- Communicating over network sockets
- Filtering Data
- Compress and Decompress data
- Writing objects to streams

Java I/O

The **java.io** package consists of input and output streams used to read and write data to files or other input and output sources.

There are 3 categories of classes in java.io package:

Input Streams

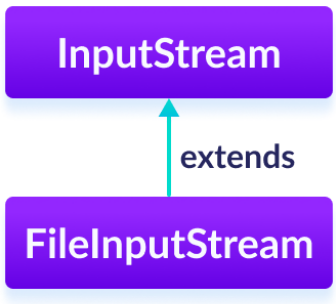
It is an abstract super-class of the **java.io** package and is used to read the data from an input source. In other words, reading data from files or from a keyboard, etc. We can create an object of the input stream class using the new keyword. The input stream class has several types of constructors.

Since `InputStream` is an abstract class, it is not useful by itself. However, its sub-classes can be used to read data.

Input streams are opened implicitly as soon as it is created. To close the input stream, we use a close() method on the source object.

In order to use the functionality of `InputStream`, we can use its sub-classes. Some of them are:

- FileInputStream
- ByteArrayInputStream
- ObjectInputStream



Create an InputStream

```
InputStream f = new FileInputStream("input.txt");
```

Here, we have created an input stream using `FileInputStream`. It is because `InputStream` is an abstract class. Hence we cannot create an object of `InputStream`.

Note: We can also create an input stream from other subclasses of `InputStream`.

Methods of InputStream

The `InputStream` class provides different methods that are implemented by its subclasses. Here are some of the commonly used methods:

- `read()` - reads one byte of data from the input stream
- `read(byte[] array)` - reads bytes from the stream and stores in the specified array
- `available()` - returns the number of bytes available in the input stream
- `mark()` - marks the position in the input stream up to which data has been read
- `reset()` - returns the control to the point in the stream where the mark was set
- `markSupported()` - checks if the `mark()` and `reset()` method is supported in the stream
- `skips()` - skips and discards the specified number of bytes from the input stream
- `close()` - closes the input stream.

▼ Example: InputStream Using FileInputStream

Suppose we have a file named **input.txt** with the following content.

```
This is a line of text inside the file.
```

Reading this file using `FileInputStream` (a subclass of `InputStream`).

```
import java.io.FileInputStream;
import java.io.InputStream;

class Main {
  public static void main(String args[]) {

    byte[] array = new byte[100];


      InputStream input = new FileInputStream("input.txt");

      System.out.println("Available bytes in the file: " + input.available());

      // Read byte from the input stream
      input.read(array);
      System.out.println("Data read from the file: ");

      // Convert byte array into string
      String data = new String(array);
      System.out.println(data);

      // Close the input stream
      input.close();


  }
}
```

Output

```
Available bytes in the file: 39
Data read from the file:
This is a line of text inside the file
```

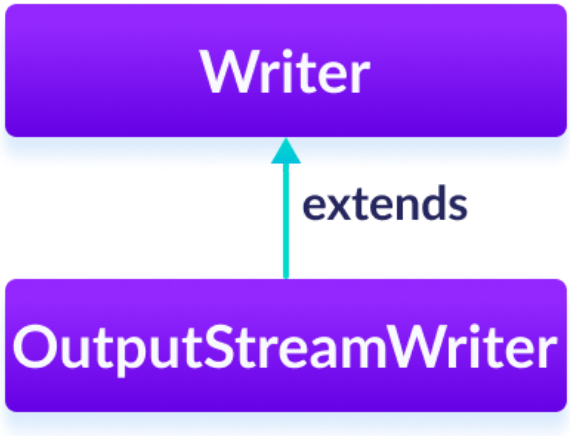
▼ Output Streams.

OutputStream: OutputStream is an abstract class of Byte Stream that describes stream output and it is used **for writing data to a file, image, audio, etc**. Thus, OutputStream writes data to the destination one at a time.

Subclasses of OutputStream

In order to use the functionality of `OutputStream`, we can use its subclasses. Some of them are:

- FileOutputStream
- ByteArrayOutputStream
- ObjectOutputStream



Create an OutputStream

In order to create an `OutputStream`, we must import the `java.io.OutputStream` package first. Once we import the package.

```
// Creates an OutputStream
OutputStream object = new FileOutputStream();
```

Here, we have created an object of output stream using `FileOutputStream`. It is because `OutputStream` is an abstract class, so we cannot create an object of `OutputStream`.

Methods of OutputStream

The `OutputStream` class provides different methods that are implemented by its subclasses. Here are some of the methods:

- `write()` - writes the specified byte to the output stream

- `write(byte[] array)` - writes the bytes from the specified array to the output stream
- `flush()` - forces to write all data present in output stream to the destination
- `close()` - closes the output stream

Example: OutputStream Using FileOutputStream

```
import java.io.FileOutputStream;
import java.io.OutputStream;

public class Main {

    public static void main(String args[]) {
        String data = "This is a line of text inside the file.";

            OutputStream out = new FileOutputStream("output.txt");

            // Converts the string into bytes
            byte[] dataBytes = data.getBytes();

            // Writes data to the output stream
            out.write(dataBytes);
            System.out.println("Data is written to the file.");

            // Closes the output stream
            out.close();



    }
}
```

To write data to the **output.txt** file, we have implemented these methods

```
output.write();      // To write data to the file
output.close();      // To close the output stream
```

OUTPUT

```
This is a line of text inside the file.
```